FECAP

FUND. 1902

Lembra??

Considere o alfabeto:

$$\Sigma = \{a, b\}$$

e a linguagem:

$$L = \{w \mid w \text{ começa com } a\}$$

a) Escreva todas as cadeias de comprimento 1 que pertencem a L.

b) Escreva todas as cadeias de comprimento 2 que pertencem a L.

c) Escreva todas as cadeias de comprimento 3 que pertencem a L.

d) As cadeias `baa`, `abb` e `aba` pertencem à linguagem? Justifique.

Métodos e notações

Nem todas as linguagens podem ser representadas por meio de uma especificação finita.

Uma **linguagem formal** é um conjunto de cadeias construídas a partir de um alfabeto e que seguem determinadas regras. Existem diferentes maneiras de representar ou trabalhar com essas linguagens. Entre elas estão as **gramáticas**, os **reconhecedores** e os **enumeradores**. Embora estejam relacionados, cada um possui uma função diferente.

Gramática

- Uma **gramática** é um conjunto de regras que descreve **como as cadeias de uma linguagem podem ser formadas**. Podemos pensar nela como um conjunto de regras de construção: partimos de um símbolo inicial e aplicamos determinadas regras até obter uma cadeia pertencente à linguagem. Também conhecidas como **dispositivos generativos**, **dispositivos de síntese**, ou ainda dispositivos de geração de cadeias

$$L = \{a, aa, aaa, aaaa, \ldots\}$$

$$S \Rightarrow a$$

$$S \Rightarrow aS \Rightarrow aa$$

$$S \Rightarrow aS \Rightarrow aaS \Rightarrow aaa$$

Assim, a gramática **define a linguagem por meio das regras utilizadas para gerar suas cadeias.**

Reconhecedor

- Um **reconhecedor** recebe uma cadeia como entrada e verifica se ela **pertence ou não a uma determinada linguagem**.
- Diferentemente da gramática, sua principal função não é mostrar como construir as cadeias, mas analisar uma cadeia que já foi fornecida

$$L = \{a, aa, aaa, aaaa, \ldots\}$$

se o reconhecedor receber:

$$aaa$$

ele verifica se essa cadeia segue as regras da linguagem.
Como possui apenas letras a e pelo menos uma ocorrência, ela é aceita.
Por outro lado, se receber:

$$aba$$

a cadeia será rejeitada, pois não pertence à linguagem

Os **autômatos** são exemplos importantes de modelos que podem ser utilizados como reconhecedores de linguagens

FUND. 1902 FECAP

Enumerador

Um **enumerador** é um mecanismo capaz de **produzir, uma a uma, as cadeias que pertencem a uma linguagem**.

$$L = \{a, aa, aaa, aaaa, \ldots\}$$

numerador poderia produzir:

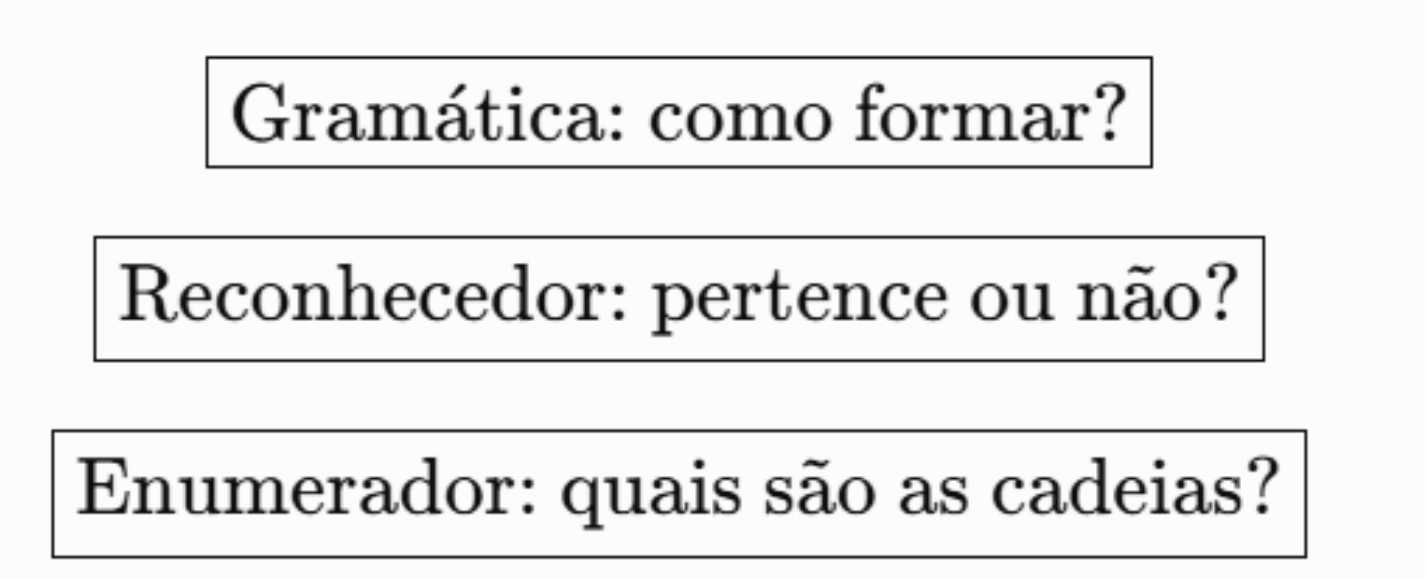
$a, aa, aaa, aaaa, aaaaa, \ldots$

Como essa linguagem é infinita, o processo poderia continuar indefinidamente.
O objetivo do enumerador não é receber uma cadeia e decidir se ela é válida, mas **produzir sistematicamente as cadeias da linguagem**.
Em uma linguagem finita, o enumerador poderia eventualmente listar todos os seus elementos. Em uma linguagem infinita, ele continua produzindo novas cadeias sem chegar ao fim.

FUND. 1902

FECAP

Resumo



Na prática, as gramáticas e os reconhecedores podem ser empregados na definição de linguagens infinitas, ao contrário das enumerações. E, ainda possibilitam uma percepção melhor da estrutura sintática inerente às sentenças das linguagens por eles definidas.

FUND. 1902

FECAP

Equivalência

Duas gramáticas podem ser diferentes, mas são ditas **equivalentes** se as duas condições abaixo forem verificadas simultaneamente:

- Toda cadeia gerada pela primeira gramática é também gerada pela segunda gramática.
- Toda cadeia gerada pela segunda gramática é também gerada pela primeira gramática.

Dois reconhecedores podem ser diferentes, mas são ditos **equivalentes** se as duas condições abaixo forem verificadas simultaneamente:

- Toda cadeia aceita pelo primeiro reconhecedor é também aceita pelo segundo reconhecedor.
- Toda cadeia aceita pelo segundo reconhecedor é também aceita pelo primeiro reconhecedor.

Equivalência

Diz-se que uma gramática é **equivalente** a um reconhecedor se as duas seguintes condições forem simultaneamente verificadas (lembrar que os formalismos devem definir todas as sentenças da linguagem desejada, e nenhuma outra cadeia):

1. Toda cadeia gerada pela gramática é também aceita pelo reconhecedor.
2. Toda cadeia aceita pelo reconhecedor é também gerada pela gramática.

Exemplo

Considere um sistema que reconhece os seguintes comandos:

$$\Sigma = \{Liga, Desliga, AbrirPorta, FecharPorta\}$$

Nesse caso, cada comando é considerado um **símbolo do alfabeto**.

Uma **cadeia** é uma sequência finita desses símbolos. Por exemplo:

$$w = Liga\ \ AbrirPorta\ \ FecharPorta\ \ Desliga$$

Essa é uma cadeia formada por **4 símbolos**

L={Liga Desliga,Liga AbrirPorta FecharPorta Desliga,Liga AbrirPorta FecharPort a AbrirPorta FecharPorta Desliga,…}

Podemos observar um padrão: o sistema deve **ligar**, pode **abrir e fechar a porta várias vezes**, e depois deve **desligar**

$$L = \{\text{Liga}(\text{AbrirPorta FecharPorta})^n\text{Desliga} \mid n \geq 0\}$$

Expressões Regulares

- As **Expressões Regulares (ER)** são uma forma de representar **padrões de cadeias pertencentes a uma linguagem formal**. Em vez de listar todas as cadeias que fazem parte de uma linguagem, podemos utilizar uma expressão que descreva como essas cadeias são formadas.
- Esse recurso é especialmente importante quando trabalhamos com **linguagens infinitas**, pois não seria possível listar todos os seus elementos.

Expressões regulares

Já estudamos diferentes **operações sobre linguagens**, como união, concatenação e fecho de Kleene. Essas operações permitem combinar linguagens e construir novos conjuntos de cadeias.

- Uma expressão regular é uma notação utilizada para representar uma **linguagem regular**. Em vez de listar todas as cadeias pertencentes à linguagem, descrevemos o padrão que essas cadeias devem seguir.
- Isso é especialmente útil quando a linguagem possui uma grande quantidade de cadeias ou é infinita.

Linguagem regular

De forma didática, uma **linguagem regular** é uma linguagem formal cujas cadeias seguem um padrão que pode ser descrito por uma **expressão regular** e reconhecido por um **autômato finito**.

Aqui, **regular não significa "normal" ou "comum"**. É o nome dado a uma classe específica de linguagens formais que pode ser representada por mecanismos equivalentes:

$$\text{Expressão Regular} \longleftrightarrow \text{Linguagem Regular} \longleftrightarrow \text{Autômato Finito}$$

Ou seja, se conseguimos descrever uma linguagem por uma expressão regular, existe um autômato finito capaz de reconhecê-la.

Construindo expressões regulares

Σ = {a, b}

A expressão: **ab***

descreve cadeias que começam com a e podem ser seguidas por zero ou mais ocorrências de b.

Assim, algumas cadeias pertencentes à linguagem são:

a

ab

abb

abbb

abbbb

Portanto:

L = {a, ab, abb, abbb, abbbb, ...}

Observe que a expressão regular não lista todas as possibilidades. Ela estabelece uma **regra que permite identificar quais cadeias fazem parte da linguagem**

FUND. 1902

FECAP

Símbolos mais usados

Símbolo	Leitura	O que significa	Exemplo
|	**OU**	Escolha entre alternativas	a|b → a ou b
*	**zero ou mais**	O elemento anterior pode aparecer nenhuma, uma ou várias vezes	a* → ε, a, aa, aaa, ...
+	**uma ou mais**	O elemento anterior deve aparecer pelo menos uma vez	a+ → a, aa, aaa, ...
ε	**épsilon / cadeia vazia**	Cadeia que não possui símbolos	L = {ε, a, aa, ...}
( )	**agrupamento**	Agrupa partes da expressão	(ab)* → ε, ab, abab, ...
ab	**a seguido de b**	Concatenação	ab → primeiro a, depois b

Ordem de precedência

Prioridade	Operação	Exemplo
1ª	**Parênteses ( )**	(a|b)*
2ª	**Repetição * e +**	a*, b+
3ª	**Concatenação**	ab
4ª	**União |**	a|b

Exemplo

ab|c*

1. Não tem Parênteses
2. Repetição c*

$$c^* \rightarrow \varepsilon, c, cc, ccc, cccc, \ldots$$

3. Concatenação ab
4. OU

Ou seja, uma cadeia pode ser **exatamente ab** OU ser formada por **zero ou mais c**

$$L = \{ab, \varepsilon, c, cc, ccc, cccc, \ldots\}$$

FUND. 1902

FECAP

Exemplo

ab*|c

Agora o asterisco está no b que pode ter ou não o b

L={a,ab,abb,abbb,...c}

(ab|c)*

O * está no **grupo inteiro**:

podemos repetir (escolher) ab ou c zero ou mais vezes:

L = {ε, ab, c, abab, abc, cab, cc, ...}

0 vezes: ε

1 vez: ab, c

2 vezes: abab, abc, cab, cc

3 vezes: ababab, ababc, abcab, abcc, cabab, cabc, ccab, ccc...

FUND. 1902

Exemplo

ab+ | c

O + está apenas no b.

Então b+ significa:

b, bb, bbb, ...

Logo:

ab+ gera:

ab, abb, abbb, abbbb, ...

E ainda temos | c, isto é, **OU c**.

Portanto:

L = {ab, abb, abbb, abbbb, ..., c}

Exemplo

(ab | c)+

O + está aplicado ao grupo inteiro:

Dentro do grupo temos: **ab OU c**

Como + significa **uma ou mais repetições**, precisamos escolher pelo menos uma vez entre ab e c.

Com uma repetição: **ab, c**

Com duas: **abab, abc, cab, cc**

Com três: **ababab, ababc, abcab, abcc, cabab, cabc, ccab, ccc, ...**

Portanto:

L = {ab, c, abab, abc, cab, cc, abababab, ...}

Aqui, **ε não pertence à linguagem**, porque + exige pelo menos uma repetição.

FUND. 1902

FECAP

Exemplo

ab|c+

O + está apenas no c.

Então: **c+** gera: **c, cc, ccc, cccc, ...**

A outra possibilidade é exatamente: **ab**

Portanto:

L = {ab, c, cc, ccc, cccc, ...}

Exercícios

Considerando $\Sigma = \{a, b\}$

1. **Começar com aa**
2. **Terminam com bbb**
3. **Contêm a subcadeia aabbb**

Exercícios

Considerando $\Sigma = \{a, b\}$

1. Começar com aa aa(a|b)*

2. Terminam com bbb $\boxed{(a \mid b)^* bbb}$

3. **Contêm a subcadeia aabbb** $\boxed{(a \mid b)^* aabbb(a \mid b)^*}$

Expressões Regulares aplicadas a sistemas computacionais

As **Expressões Regulares (ER)** permitem descrever padrões de cadeias pertencentes a uma linguagem. Em sistemas computacionais, esses padrões podem ser utilizados para representar quais entradas, comandos ou mensagens são considerados válidos.

Definir uma regra → representar por uma ER → receber uma entrada → verificar se corresponde ao padrão → aceitar ou rejeitar

ER teórica × Regex de programação

Por exemplo, para validar um código formado por três dígitos:

Em uma notação mais próxima da ER teórica, poderíamos escrever:
$(0 \mid 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9)(0 \mid 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9)(0 \mid 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9)$

Em uma regex de programação:

[0-9]{3}

As duas expressam a mesma ideia: **três dígitos consecutivos**.

ER teórica × Regex de programação

- As regex utilizadas na programação têm origem nas Expressões Regulares estudadas em Linguagens Formais. Enquanto a ER teórica descreve matematicamente linguagens regulares, as regex de programação incorporam notações e recursos adicionais para facilitar a busca, o reconhecimento e a validação de padrões em aplicações reais.

Nem todo recurso oferecido por uma regex moderna corresponde estritamente às expressões regulares da teoria.

ER teórica × Regex de programação

- A ER teórica trabalha principalmente com operações como: | *união*; concaten*ação*; $*$ *fecho de Kleene*
- Já os mecanismos modernos de regex oferecem atalhos e recursos co

Símbolo	Significado	Exemplo	Aceita
.	Qualquer caractere (exceto quebra de linha)	a.c	abc, a7c, a_c
^	Início da string	^abc	abcdef
$	Fim da string	abc$	123abc
[]	Classe de caracteres	[abc]	a, b ou c
[^]	Negação da classe	[^0-9]	qualquer caractere que não seja número
-	Intervalo dentro de []	[A-Z]	letras maiúsculas
|	OU	casa|carro	casa ou carro
()	Agrupamento	(ab)+	ab, abab
*	Zero ou mais ocorrências	ab*	a, ab, abb
+	Uma ou mais ocorrências	ab+	ab, abb
{n}	Exatamente n ocorrências	\d{3}	123
{n,}	Pelo menos n ocorrências	a{2,}	aa, aaa
{n,m}	Entre n e m ocorrências	a{2,4}	aa, aaa, aaaa
\	Caractere de escape	\.	procura um ponto (.) literal

Nem todo recurso oferecido por uma regex moderna corresponde estritamente às expressões regulares da teoria.

As regex utilizadas na programação nasceram das expressões regulares teóricas, mas alguns mecanismos modernos adicionaram recursos, como referências a grupos capturados, que permitem representar padrões que não são linguagens regulares no sentido clássico da teoria.

Exemplo 1

aa(a|b)*

Algumas cadeias da linguagem:

$$L = \{aa, aaa, aab, aaaa, aaab, aaba, aabb, \ldots\}$$

aa → pertence

aab → pertence

aababa → pertence

aba → não pertence

baa → não pertence

FUND. 1902

FECAP

Exemplo 1

aa(a|b)*

```
import re

padrao = r"aa(a|b)*"

cadeia = input("Digite uma cadeia: ")

if re.fullmatch(padrao, cadeia):
   print("Cadeia pertence à linguagem")
else:
   print("Cadeia não pertence à linguagem")
```

re.fullmatch() verifica se **toda a cadeia** corresponde ao padrão.
O r antes das aspas indica que aquela string é uma **raw string**, ou seja, uma **string bruta, para manter todos os caracteres.**

re.match() início
re.search() qualquer lugar
re.fullmatch() cadeia inteira

Exemplo 2

- Agora queremos uma linguagem formada por códigos que começam com A e possuem **exatamente três dígitos**:

A123
A007

$$A(0|1|2|3|4|5|6|7|8|9)(0|1|2|3|4|5|6|7|8|9)(0|1|2|3|4|5|6|7|8|9)$$

Exemplo 2

- Agora queremos uma linguagem formada por códigos que começam com A e possuem **exatamente três dígitos**:

A123
A007

```
import re

padrao = r"A[0-9]{3}"
codigo = input("Digite o código: ")

if re.fullmatch(padrao, codigo):
    print("Código válido")
else:
    print("Código inválido")
```

Exemplo 3

Regex

^E[1-3]{3}$

Quais cadeias aceitas?

Exemplo 3

^E[1-3]{3}$

[1-3][1-3][1-3]

^ → início da string. E → deve começar com a letra E. [1-3] → aceita apenas os dígitos **1**, **2** ou **3**.

{3} → exatamente três ocorrências. $ → fim da string.

Válidos

E111

E123

Inválidos

E145 (4 e 5 não são permitidos)

E12 (apenas dois dígitos)

FUND. 1902

FECAP

Exercício 1

Um sistema de automação aceita apenas os seguintes comandos:

LIGAR:LAMPADA
DESLIGAR:LAMPADA

a) Construa a expressão regex que representa os comandos permitidos.
b) Desenvolva um programa em Python que receba um comando e informe se ele pertence à linguagem definida.

(LIGAR|DESLIGAR):LAMPADA

^(LIGAR|DESLIGAR):LAMPADA$

```
import re

comando = input("Digite o comando: ")

if re.fullmatch(r"(LIGAR|DESLIGAR):LAMPADA",
comando):
    print("Comando válido!")
else:
    print("Comando inválido!")
```

Exercício 2

Um sensor envia mensagens no seguinte formato: TEMP:25

A mensagem deve começar obrigatoriamente com TEMP: e possuir exatamente dois dígitos após os dois-pontos.

Exemplos válidos:

TEMP:08
TEMP:25
TEMP:99

Exemplos inválidos:

TEMP:5
TEMP:123
TEM:25

a) Represente o padrão por meio de uma regex.
b) Crie um programa Python que valide a mensagem recebida

TEMP:(0|1|2|3|4|5|6|7|8|9)
(0|1|2|3|4|5|6|7|8|9)

^TEMP:\d{2}$

Trecho	Significado
^	Início da string
TEMP:	Texto obrigatório
\d	Um dígito (0 a 9)
{2}	Exatamente dois dígitos
$	Fim da string

```
import re

mensagem = input("Digite a mensagem: ")

if re.fullmatch(r"TEMP:\d{2}", mensagem):
    print("Mensagem válida!")
else:
    print("Mensagem inválida!")
```

Exercício 3

Um sistema define que o identificador de um usuário deve começar com U, seguido de exatamente quatro dígitos.

U1234 → válido
U0007 → válido
A1234 → inválido
U123 → inválido

a) Escreva a expressão regular correspondente.
b) Utilize re.fullmatch() para verificar o identificador informado pelo usuário.

U(0|1|2|3|4|5|6|7|8|9)
(0|1|2|3|4|5|6|7|8|9)
(0|1|2|3|4|5|6|7|8|9)
(0|1|2|3|4|5|6|7|8|9)

^U\d{4}$

```
import re

identificador = input("Digite o identificador: ")
if re.fullmatch(r"U\d{4}", identificador):
    print("Identificador válido!")
else:
    print("Identificador inválido!")
```

Exercício 4

Um dispositivo IoT pode enviar informações de temperatura ou umidade no formato:

TEMP:25

ou

UMID:70

- O valor deve possuir exatamente dois dígitos.

a) Construa uma única expressão regular para representar os dois tipos de mensagens.
b) Desenvolva um programa Python que receba a mensagem e informe se ela é válida.

(TEMP|UMID):(0|1|2|3|4|5|6|7|8|9)(0|1|2|3|4|5|6|7|8|9)

^(TEMP|UMID):\d{2}$

```
import re

mensagem = input("Digite a mensagem: ")
if re.fullmatch(r"(TEMP|UMID):\d{2}", mensagem):
    print("Mensagem válida!")
else:
    print("Mensagem inválida!")
```

Exercício 5

Um sistema aceita comandos formados por uma ação e um dispositivo. As ações possíveis são:

LIGAR
DESLIGAR

Os dispositivos possíveis são:

LUZ
VENTILADOR

O formato da mensagem é:

ACAO:DISPOSITIVO

Exemplos válidos:

LIGAR:LUZ
DESLIGAR:LUZ
LIGAR:VENTILADOR
DESLIGAR:VENTILADOR

Faça:
a) Determine a regex que representa a linguagem desses comandos.
b) Desenvolva o programa responsável pela validação.

(LIGAR|DESLIGAR):(LUZ|VENTILADOR)

^(LIGAR|DESLIGAR):(LUZ|VENTILADOR)$

```
import re

comando = input("Digite o comando: ")
if
re.fullmatch(r"(LIGAR|DESLIGAR):(LUZ|VENTILADOR)",
comando):
    print("Comando válido!")
else:
    print("Comando inválido!")
```

Exercício 6

Um sistema utiliza tópicos no formato:

casa/sala/temperatura

casa/quarto/temperatura

casa/cozinha/temperatura

Somente os ambientes sala, quarto e cozinha são aceitos.

Faça:
a) Construa a regex que represente os tópicos válidos.
b) Faça um programa em Python que receba um tópico e informe se ele é válido.

casa/(sala|quarto|cozinha)/temperatura

^casa/(sala|quarto|cozinha)/temperatura$

```
import re

topico = input("Digite o tópico: ")

if  re.fullmatch(r"casa/(sala|quarto|cozinha)/temperatura",
topico):
    print("Tópico válido!")
else:
    print("Tópico inválido!")
```

Verificando...

Função	O que faz	Quando usar
re.fullmatch()	Verifica se **toda a string** segue o padrão.	Validação.
re.findall()	Procura **todas as ocorrências** do padrão dentro da string.	Extração de informações.

`re.fullmatch()`
Serve para responder:
"O texto inteiro está no formato esperado?"

`re.findall()`
Serve para responder:
"Quais partes do texto correspondem ao padrão?"

Exemplo

```
import re

texto = "Sensor 15 temperatura 27 umidade 63"

print(re.findall(r"\d+", texto))
```

Exemplo com arquivo de texto

Relatório de Monitoramento IoT

Sensor 15 registrou TEMP 27 graus às 08:30.
Sensor 18 registrou UMID 63% às 08:31.
Sensor 21 registrou TEMP 29 graus às 08:32.
Sensor 22 registrou UMID 58% às 08:33.

Equipamento 105 desligado para manutenção.
Equipamento 204 voltou a operar normalmente.

A leitura PRESSAO 1013 hPa foi registrada, mas não será pesquisada.
O sensor 30 registrou TEMP 31 graus e UMID 70%.

Código do lote: A4589
Número da Ordem de Serviço: 2026

FUND. 1902

FECAP

```
import re

# Abre o arquivo para leitura
with open("mensagem.txt", "r", encoding="utf-8") as arquivo:
    texto = arquivo.read()

# Procurar as palavras TEMP ou UMID
palavras = re.findall(r"TEMP|UMID", texto)

# Procurar todas as sequências numéricas
numeros = re.findall(r"\d+", texto)

print("Palavras encontradas:")
print(palavras)

print("\nNúmeros encontrados:")
print(numeros)
```

Bibliografia

Linguagens Formais: Teoria e Conceitos
M.V.M. Ramos, J.J. Neto e I.S. Vega
Editora Blucher, 2023
ISBN 9786555067163

FECAP

FUND. 1902